CSC-239 *Java Programming*

Chapter 11: Inheritance and Polymorphism (Part 2)

This lecture is being recorded.

October 20, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Project 1 was due Oct. 15

- Project 1 was due on Wednesday, October 15.
- As stated in the *Syllabus*, the grade penalty for turning in a project late is:

  1% per day (after a 1-day grace period), up to a maximum grade penalty of 20%.

  Any project that is 21 days late (or more) receives a grade of ZERO.
- What this means is: if you need a few extra days to do a good job on your project, then the late penalty will not be very much.
- However, if you need significantly more time than that, you should meet with the instructor to get some help.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Remember previous Lecture

We introduced the concept of **Inheritance**:

- One class is a *specialization* of another class.
- We say that there exists an "**IsA**" relationship between the two classes: for example:

  A Car **IsA** Vehicle.

  A Student **IsA** Person.

Syntax for Inheritance

The Syntax for defining a SubClass:

```
public class SuperClass {
        . . .
}

class SubClass extends SuperClass
{
        . . .
}
```

The *IsA* Relationship

- An object of the subclass **IS** an object of the superclass.
- ***Superclass*** is also referred to as the
  - Parent class
  - Base class
- ***Subclass*** is also referred to as the
  - Child class
  - Extended class
  - Derived class

CAUTION: Do not confuse the ***IsA*** relationship with the ***HasA*** relationship (see Chapter 10).

Overriding vs. Overloading

Be careful not to confuse these two.

- "Override" a method of the superclass:
  - A subclass method with the same name and parameter types as the superclass method (same signature).
  - The final method selection is done at **runtime**.
  - Overriding is *not* the same as Overloading.
- "Overload" a method:
  - Two methods with the same name but *different* parameter types (different signatures).
  - Method selection is done at **compile time**.

OverrideTest.java

```
public class OverrideTest {
    public static void main(String[] args) {
        A a = new A();
        a.p(10);
        a.p(10.0);
    }
}
class B{
    public void p(double i){
        System.out.println(i * 2);
    }
}
class A extends B {
    public void p(double i){  // A.p(double) overrides
                              // B.p(double)
        System.out.println(i);
    }
}
```

OverloadTest.java

```
public class OverloadTest {
    public static void main(String[] args) {
        A a = new A();
        a.p(10);
        a.p(10.0);
    }
}
class B{
    public void p(double i){
        System.out.println(i * 2);
    }
}
class A extends B {
    public void p(int i){      // A.p(int)      overloads
                               // B.p(double)
        System.out.println(i);
    }
}
```

OverrideTest vs. OverloadTest

`OverrideTest` output:

```
run:
10.0
10.0
```

`OverloadTest` output:

```
run:
10
20.0
```

What is Happening?

- When we run the **OverrideTest** class, both method calls

  `a.p(10)` and

  `a.p(10.0)`

  invoke the **`p(double)`** method in class A.

- When you run the **OverloadTest** class,

  `a.p(10)` invokes the method in class A, and

  `a.p(10.0)` invokes the method in class B.

The Object Class and Its Methods

- Every class in Java is descended from the java.lang.Object class.
- If no inheritance is specified when a class is defined, the superclass of the class is Object:

```
public class Circle {...}
```

is equivalent to

```
public class Circle extends Object {...}
```

The toString() method in Object

The `toString()` method returns a string representation of the object.

```
Fruit f = new Fruit();
System.out.println(f.toString());
```

The code above displays something like

```
Fruit@15037e5
```

This message is not very helpful or informative. Usually you should override the toString method so that it returns a meaningful string representation of the object.

Polymorphism Example

```
public class PolymorphismDemo {
  public static void main(String[] args) {
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
  }
  public static void m(Object x) {
    System.out.println(x.toString());
  }
}

class GraduateStudent extends Student {     }

class Student extends Person {
  public String toString() {  return "Student"; }
}

class Person extends Object {
  public String toString() {  return "Person";  }
}
```

Polymorphism Example: Output

```
Student
Student
Person
java.lang.Object@378bf509

Process finished with exit code 0
```

Same Method `m`, Different Argument Type

- In the example:
  - `Person` extends `Object`
  - `Student` extends `Person`
  - `GraduateStudent` extends `Student`
- Method **`m`** takes a parameter of the **`Object`** type. You can invoke it with any object.
- An object of a subtype can be used wherever its supertype value is required. This feature is known as *polymorphism*.

Dynamic (runtime) Type Binding

- When executing method `m(Object x)` the argument x's `toString` method is invoked.
  The x object may be an instance of `GraduateStudent`, `Student`, `Person`, or `Object`.
- Classes `Student`, `Person`, and `Object` each have their own implementation of the `toString` method.
- Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime. This capability is known as *dynamic binding*.

Dynamic Binding

- Object o is an object of the class C_1.
- Object o invokes method p.
- The JVM searches the inheritance chain for the required method, starting at class C_1.
- Once an implementation is found, the search stops and the first-found implementation is invoked.

